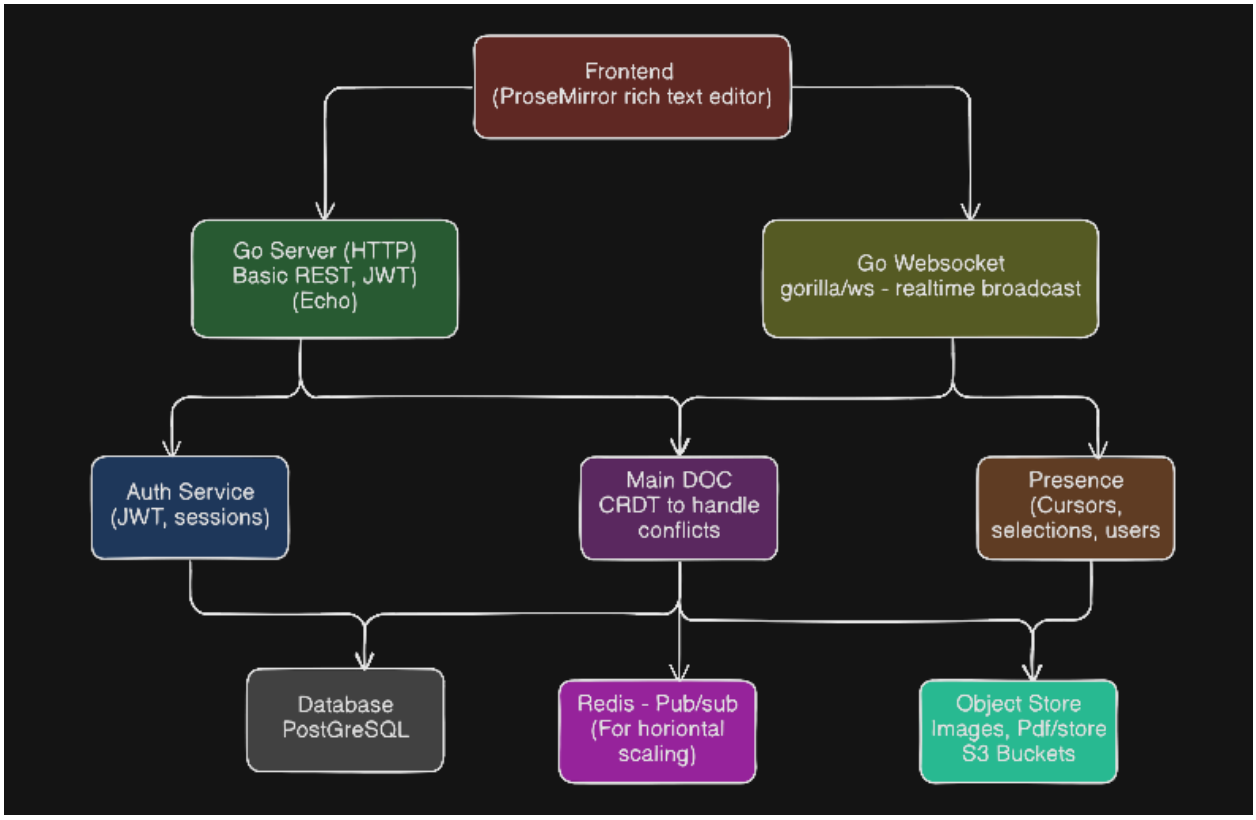
Making a Google Doc From Scratch

Project Architecture



1. Summary & Goals

This project aims to build a highly scalable, real-time collaborative rich-text editor using a modern Go backend and Next.js frontend. The core technical challenge lies in synchronizing state across multiple active clients without data loss or race conditions.

2. Final Technology Stack

Layer	Choice	Rationale
Frontend Framework	Next.js	Industry standard React framework, excellent for SEO, structured routing, and component management.

Backend Language	Go (Golang)	High performance, excellent concurrency model (goroutines/channels) specifically suited for concurrent WebSocket handling.
HTTP Routing	Echo	Clean API, runs on the standard `net/http` library, ensuring compatibility with standard WebSocket packages.
WebSockets	gorilla/websocket	The most mature and widely-used standard for WebSockets in Go.
Conflict Resolution	Yjs (CRDT)	Mathematical guarantees against data divergence. The backend acts as a fast routing pipe for binary updates.
Editor UI	ProseMirror + y-prosemirror	Battle-tested rich text editor with first-class, out-of-the-box bindings for Yjs.
Database	PostgreSQL	ACID compliant, perfect for relational data (users/permissions) and handles Yjs binary blobs via `BYTEA` columns natively.
Pub/Sub Cache	Redis	Essential for horizontal scaling to route real-time WebSocket messages between different Go server instances

Object Storage	Amazon S3 (or MinIO)	Offloads large media files (images, PDFs) from the main database, keeping the sync fast and lightweight..

3. Microservices Architecture

To ensure scalability and separation of concerns, the backend is architected into the following microservices, separated by failure domains and scaling profiles:

1. **API Gateway & Authentication Service (Stateless):** Handles user registration, login, JWT token issuance, rate limiting, and acts as the front door. Scales horizontally easily based on standard HTTP web traffic.
2. **Document Management Service (Stateless):** Manages traditional RESTful CRUD operations. Handles creating, deleting, and renaming documents, fetching metadata, as well as updating user permissions (Viewer, Editor, Owner). Keeping this separate prevents heavy database queries from blocking real-time WebSocket threads.
3. **Real-Time Sync Service (Stateful - The Core):** The WebSocket hub. Maintains active, long-lived connections with clients, holds active document rooms in memory, receives binary Yjs updates, and broadcasts them to everyone in the room. This service is memory-heavy and utilizes Redis Pub/Sub to communicate state changes across multiple instances of itself.
4. **Persistence Worker (Background Daemon):** Listens to the stream of document updates and periodically (e.g., every 15-30 seconds or on client disconnect) saves a compacted snapshot of the Yjs state to PostgreSQL. This completely decouples the "hot path" (rapid typing) from the "cold path" (disk I/O).
5. **Presence Service (Awareness):** Handles high-frequency, transient data like user cursors, text selections, and active user lists. Utilizing Yjs's awareness protocols, this module isolates rapid visual updates (like mouse movements) so they can be broadcast instantly to clients without ever being permanently saved to the database.
6. **Object Store Service (S3):** Adding S3 for images and PDFs makes this a true Google Docs competitor. Storing large media in PostgreSQL is a bad practice, so offloading that to an S3 bucket and just keeping the image URLs in the Yjs document state keeps the application lightning fast.

4. The CRDT Strategy

This project employs a dual-track approach to conflict resolution to maximize both product stability and educational depth.

- **Production Approach (Yjs):** For the actual working clone, Yjs operates on the client side. The Go backend does not compute document merges; it blindly and rapidly forwards Yjs binary update blobs. This guarantees flawless rich-text synchronization without the astronomical complexity of building a rich-text engine from scratch.
- **The RGA Part (Learning Track):** To truly master the mathematics of CRDTs, a side project will be built directly in Go. An RGA (Replicated Growable Array) will be implemented from scratch. This standalone Go package will handle plain-text distributed synchronization, serving as a masterclass in distributed data structures, pointers, and Go-based memory management.

5. Implementation Roadmap

- **Phase 1: Foundation (Auth & API):** Set up the Echo server, PostgreSQL schemas (Users, Documents), and build JWT authentication and basic REST endpoints.
- **Phase 2: The UI Layer:** Initialize the Next.js application, implement ProseMirror, and create the static document editing interface.
- **Phase 3: Real-Time Sync (Single Server):** Implement `gorilla/websocket`. Connect Next.js/y-websocket to the Go server. Achieve real-time typing between two browser windows connected to the same Go instance.
- **Phase 4: Database Persistence:** Build the background worker to periodically save the live, in-memory Yjs binary blob to the PostgreSQL `BYTEA` column.
- **Phase 5: Scale-Out (Redis):** Introduce Redis Pub/Sub. Spin up two Go Sync servers on different ports and verify that a client on Server A can see the keystrokes of a client on Server B.
- **Phase 6: The RGA Sandbox:** Step away from the main project to build the custom RGA plain-text engine in pure Go to complete the learning.